DSA — Day 2

Expanded Lecture-Remembering Notes • Java Code • Before/After Optimisations

> Built from the uploaded Day 2 lecture transcript. The explanations preserve the instructor's teaching flow and memorable phrases, while code formatting and a few clarifications are added for revision.

Clickable Contents

1. Day 1 recap and the lecturer's learning method
2. Bubble sort: philosophy and dry run
3. Bubble sort: first Java implementation
4. Improved bubble sort: shrink the inner loop
5. More improved bubble sort: swap flag
6. Insertion technique before insertion sort
7. Insertion sort: complete Java implementation
8. Bubble sort vs insertion sort and complexity
9. Linked lists: concept, node and types
10. Singly linked list: core Java implementation
11. Linked list operations and edge cases
12. Revision sheet, common traps and practice

1. Day 1 recap and the lecturer's learning method

Sir begins by recapping the previous session: data structures and ADTs, efficiency analysis, asymptotic analysis, linear search, binary search, recursion, and selection sort. The new focus is bubble sort, insertion sort, and then linked lists.

Lecture-style takeaway: "First understand the problem, then put it in algorithmic way, and then convert that logic into code." The goal is not merely to use Java's built-in methods; it is to understand what happens behind them.

- **Paperwork first:** dry-run an algorithm on paper at least twice.
- **Talk to yourself while coding:** say what each line is doing so that you follow the logic, not just the syntax.
- **Analyse the implementation you actually wrote:** a small code change may improve real running time or best-case behaviour without changing the worst-case Big-O.

The transcript uses `Arrays.sort()` and `Arrays.toString()` in examples for convenience. These are Java library helpers; they are not the sorting algorithm being taught.

2. Bubble sort: philosophy and dry run

Sir's memory hook is the bubble in water: as you move through the array, the larger value gradually moves towards the end. Bubble sort compares **two consecutive (adjacent) elements**. If the left element is greater than the right element, swap them; otherwise, leave them as they are.

In his words, the algorithm is essentially: "compare, compare consecutive elements; if the left-hand side element is greater than the right-hand side element, then swap."

Example input: `[6, 4, 2, 8, 3, 1]`

Pass 1: 6 and 4 swap → `[4, 6, 2, 8, 3, 1]`; 6 and 2 swap → `[4, 2, 6, 8, 3, 1]`; 6 and 8 do not swap; 8 and 3 swap; 8 and 1 swap. End of pass: `[4, 2, 6, 3, 1, 8]`. The largest element, 8, has reached the end.

Repeat passes. After each full pass, the largest remaining unsorted value settles into its final position at the right. For `n` elements, the simple version needs at most `n – 1` passes.

> The word "bubble" is the memory hook, not a strict scientific description: the key algorithmic fact is that adjacent comparisons and swaps move larger values rightward.

3. Bubble sort: first Java implementation

Sir separates the job into two loops. The outer loop counts the passes; the inner loop performs one pass by comparing adjacent elements. For a pass, `j` must stop at `length - 2`, because the code also reads `a[j + 1]`.

```
static void bubbleSort(int[] a) {
    int n = a.length;

    for (int i = 0; i < n - 1; i++) {       // n - 1 passes
        for (int j = 0; j < n - 1; j++) {   // one complete pass
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

Important index note: `j < n - 1` is not the same as `j <= n - 1`. If `j` reaches `n - 1`, `a[j + 1]` becomes `a[n]`, which is out of bounds.

Sir's first implementation deliberately scans the whole inner range on every pass, even though the last few values are already in their final positions. This gives a straightforward baseline that can be optimised.

4. Improved bubble sort: shrink the inner loop

After the first pass, the largest element is already at the last index, so comparing it again in the next pass is unnecessary. After the second pass, the two largest elements are settled at the end. Therefore, reduce the inner-loop limit by `i`.

```
static void bubbleSortImproved(int[] a) {
    int n = a.length;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

What changed? Only the inner-loop condition: from `j < n - 1` to `j < n - 1 - i`. The outer loop still controls the passes.

For six elements, the first version does 5 comparisons in each of 5 passes = 25 comparisons. The improved version does 5 + 4 + 3 + 2 + 1 = 15 comparisons. Both remain $O(n^2)$ in the usual worst case, but the improved version avoids

needless comparisons.

General comparison count for this version: `(n–1) + (n–2) + ... + 1 = n(n–1)/2`, which grows quadratically.

5. More improved bubble sort: swap flag

Sir then asks: what if the array is already sorted? Even the improved version still performs all planned passes. If a pass performs no swaps, that tells us the adjacent values were already in order, so the array is sorted and we can stop.

Use a Boolean flag. At the beginning of each pass, set it to `false`. Whenever a swap happens, set it to `true`. At the end of the pass, if it is still `false`, `break`.

```
static void bubbleSortOptimized(int[] a) {
    int n = a.length;

    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false; // reset for each pass

        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
                swapped = true;
            }
        }

        if (!swapped) {
            break; // no swap means already sorted
        }
    }
}
```

Why reset the flag? Each pass must independently answer: “Did this pass perform any swap?” If the flag is not reset, a swap from an earlier pass could prevent the algorithm from detecting that a later pass did no work.

Complexity depends on the implementation: for an already-sorted array, this flag version performs one pass of `n–1` comparisons, so best-case time is O(n). Average and worst case remain $O(n^2)$. Without the flag, even an already-sorted array takes $O(n^2)$ comparisons.

> Interview phrasing: say which implementation you mean. Do not claim every bubble-sort implementation has an O(n) best case.

6. Insertion technique before insertion sort

Sir deliberately pauses before teaching the full insertion sort: “I’m not teaching you insertion sort; I’m only talking about the insertion technique first.” The key idea is that the left portion is already sorted and the last element is the one that is out of place.

Example: `[1, 3, 4, 6, 8, 2]`. Temporarily take `2` out and store it in `temp`. Compare it with earlier elements from right to left. Since 8, 6, 4, and 3 are all greater than 2, shift each one position to the right. Stop when you reach 1, because 1 is not greater than 2. Put 2 after 1.

Result: `[1, 2, 3, 4, 6, 8]`.

Sir’s key wording: “I do not say swap it; I said shift it.” This is the important difference from bubble sort. In insertion sort, hold the key in a temporary variable, shift larger earlier values right, and finally insert the key into the gap.

If the key is smaller than every earlier element, `j` can become `-1`. That is a valid stopping condition for the loop logic, but `a[-1]` must never be accessed. Use `j >= 0 && a[j] > key` so Java’s short-circuit `&&` prevents the array access when `j` is `-1`.

7. Insertion sort: complete Java implementation

The lecture first demonstrates the insertion technique on the last element. The full insertion sort repeats that technique for each element, starting from index 1. At each iteration, the part to the left of `i` is treated as sorted.

```
static void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i];      // element to insert
        int j = i - 1;       // last index of sorted portion

        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j]; // shift right, not swap
            j--;
        }

        a[j + 1] = key;      // insert into the gap
    }
}
```

Dry run: [5, 2, 4, 1]

Start: `[5, 2, 4, 1]`. For `i = 1`, key = 2; shift 5 right and insert 2 → `[2, 5, 4, 1]`. For `i = 2`, key = 4; shift 5 right and insert 4 → `[2, 4, 5, 1]`. For `i = 3`, key = 1; shift 5, 4, and 2 right, then insert 1 → `[1, 2, 4, 5]`.

The outer loop is not sorting the entire array from scratch each time. It expands a sorted prefix one element at a time.

8. Bubble sort vs insertion sort and complexity

Algorithm / version	Main idea	Best case	Average / worst case	Extra space
Bubble, basic	Adjacent compare + swap	$O(n^2)$	$O(n^2)$	O(1)
Bubble, shrinking range	Skip settled suffix	$O(n^2)$	$O(n^2)$	O(1)
Bubble, swap flag	Stop if a pass has no swaps	O(n)	$O(n^2)$	O(1)
Insertion sort	Shift larger values; insert key	O(n)	$O(n^2)$	O(1)

Insertion sort's best case is O(n) because an already sorted array requires only one comparison per inserted element and no shifting. Its worst case is $O(n^2)$, for example when sorting in ascending order an array initially arranged in descending order.

Big-O is an asymptotic growth description, not an exact stopwatch reading. Fewer comparisons often mean better actual running time for a particular input, but the same $O(n^2)$ label can cover implementations with different constant factors and different best cases.

9. Linked lists: concept, node and types

After sorting, the lecture moves to linked lists. The transcript frames a linked list as a list of nodes connected together. A node has two parts in a basic singly linked list: the data and a reference to the next node.

```
[data | next] -> [data | next] -> [data | null]
head
```

head points to the first node. The last node's `next` is `null` in a standard linear singly linked list. Unlike an array, the nodes need not sit next to each other in memory; the references connect them.

The transcript discusses these types:

- **Singly linear linked list:** each node points to the next; the last points to `null`.
- **Singly circular linked list:** like singly linked, but the last node points back to the first node.
- **Doubly linked list:** nodes have links to both previous and next nodes.

- **Doubly circular linked list:** previous/next links form a circle; the last points forward to the first, and the first points backward to the last.

The abstract operations stay conceptually similar, while the internal representation varies by type. The lecture emphasises that you should learn operations one by one and understand how each is implemented.

10. Singly linked list: core Java implementation

The transcript's object-oriented approach uses a `Node` class and a `LinkedList`/list class that owns the head reference and methods such as display, add first, add last, delete first, and delete last. Below is a clean, complete teaching implementation of the same basic design.

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

class SinglyLinkedList {
    private Node head;

    public void display() {
        Node temp = head;
        while (temp != null) {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }
        System.out.println("null");
    }

    public void addFirst(int value) {
        Node newNode = new Node(value);
        newNode.next = head;
        head = newNode;
    }

    public void addLast(int value) {
        Node newNode = new Node(value);

        if (head == null) {
            head = newNode;
            return;
        }

        Node temp = head;
        while (temp.next != null) {
            temp = temp.next;
        }
        temp.next = newNode;
    }

    public void deleteFirst() {
        if (head == null) {
            System.out.println("List is empty");
            return;
        }
        head = head.next;
    }

    public void deleteLast() {
        if (head == null) {
            System.out.println("List is empty");
            return;
        }

        if (head.next == null) { // exactly one node
            head = null;
            return;
        }
```

```
        Node temp = head;
        while (temp.next.next != null) {
            temp = temp.next;
        }
        temp.next = null;
    }
}

public class Main {
    public static void main(String[] args) {
        SinglyLinkedList list = new SinglyLinkedList();
        list.addLast(10);
        list.addLast(20);
        list.addLast(30);
        list.display();      // 10 -> 20 -> 30 -> null

        list.addFirst(5);
        list.display();      // 5 -> 10 -> 20 -> 30 -> null

        list.deleteLast();
        list.display();      // 5 -> 10 -> 20 -> null

        list.deleteFirst();
        list.display();      // 10 -> 20 -> null
    }
}
```

This is an expanded, cleaned-up implementation to make the transcript's concepts easy to run and revise; method names and formatting may differ from the instructor's exact live code.

11. Linked list operations and edge cases

Add first

Create a new node, point its `next` to the current `head`, then update `head` to the new node. Order matters: if you overwrite `head` before saving the old connection, you can lose access to the existing list.

Add last

If the list is empty, the new node becomes `head`. Otherwise, traverse until `temp.next == null`, then connect `temp.next` to the new node.

Delete first

If the list is empty, handle that case. Otherwise, move `head` to `head.next`. The old first node is no longer reachable from the list.

Delete last

There are three cases: (1) empty list — nothing to delete; (2) one node — set `head = null`; (3) two or more nodes — stop at the second-last node and set its `next` to `null`.

Why does `temp.next.next` appear? In the loop `while (temp.next.next != null)`, `temp` is kept at the second-last node when the next node is the last node. This condition is safe only after checking that the list has at least two nodes.

Common operations and typical time costs for a singly linked list without a tail pointer:

Operation	Typical time	Reason
Add first	O(1)	Only update head and one link
Delete first	O(1)	Move head forward
Display / traverse	O(n)	Visit each node
Add last	O(n)	Must find the last node
Delete last	O(n)	Must find the second-last node

The transcript also mentions insertion/deletion at a given position and the possibility of exceptions or invalid positions. Those methods need explicit checks for empty lists, invalid indexes, and the special case of the first node.

12. Revision sheet, common traps and practice

Remember in one line

- Bubble sort: compare adjacent values; swap when left > right; a large value moves to the end on each pass.
- Improved bubble sort: inner loop shrinks by `i` because the suffix is already sorted.
- More improved bubble sort: `swapped == false` after a pass means stop early.
- Insertion technique: save the key, shift larger values right, insert the key into the gap.
- Insertion sort: repeat insertion for each key from index 1 onward.
- Linked list node: data + next reference; `head` identifies the first node.
- Delete last: do not forget empty-list and one-node cases.

Common mistakes

- Using `j <= n - 1` when reading `a[j + 1]` causes an out-of-bounds access.
- Forgetting to reset `swapped = false` at the start of each bubble-sort pass.
- Saying insertion sort swaps adjacent elements like bubble sort; insertion sort shifts elements and inserts the saved key.
- Accessing `a[j]` before checking `j >= 0` in insertion sort.
- Deleting the last node without handling a one-node list.
- Assuming every improvement changes Big-O. The shrinking-range bubble sort improves the comparison count versus the basic version, but both remain $O(n^2)$.

Practice questions

1. For `[7, 3, 5, 1]`, write the array after each pass of basic bubble sort.
2. Why does the inner loop use `j < n - 1 - i` in improved bubble sort?
3. What does `swapped == false` tell us after one complete pass?
4. In insertion sort, why do we save `a[i]` in `key` before shifting?
5. What does the insertion-sort condition `j >= 0 && a[j] > key` protect against?
6. What is the best-case time complexity of flag-optimised bubble sort, and for which input?
7. Draw three nodes and show `head`, `next`, and `null` for a singly linked list.
8. What are the three cases that `deleteLast()` must handle?
9. Explain why adding at the head is O(1) but adding at the end is O(n) without a tail pointer.
10. Explain the difference between a linear singly linked list and a circular singly linked list.

Self-check answers: 2) The last `i` elements are already in final positions. 3) No swap occurred, so the array is sorted. 4) Shifting may overwrite the key's original slot. 5) It prevents reading a negative index. 6) O(n), when the array is already sorted. 8) Empty, one node, two-or-more nodes. 9) Head insertion updates a fixed number of references; tail insertion must traverse the nodes unless a tail reference is maintained. 10) Linear list ends at `null`; circular list's last node points back to the first.

Study method for future days: I'll use this same style for future transcripts: preserve Sir's memorable explanations and sequence, expand the reasoning, include runnable Java code, show before/after code when the lecture makes a change, add dry runs and complexity, and clearly label any extra clarification or cleaned-up code.